

Bayesian Methods for Neural Networks

Brunetto Daniele

Mattia Faini

Original report: 12 May 2026

English translation: 29 September 2026

Abstract

Modern deep artificial neural networks (ANNs), while offering considerable flexibility and excellent performance, suffer from substantial overparameterization that leads them to produce overly confident predictions on out-of-distribution data. This report explores the integration of Bayesian methods into deep learning. By treating model parameters as random variables, Bayesian neural networks (BNNs) can capture and quantify both aleatoric uncertainty (data noise) and epistemic uncertainty (the model’s lack of knowledge). Exact Bayesian inference and calculation of the true posterior, however, are computationally intractable in deep architectures because of their high dimensionality, making approximation necessary. This document analyzes and compares two principal methods for making uncertainty estimation scalable: variational inference through Bayes by Backprop and Monte Carlo Dropout.

Keywords: Bayesian Neural Networks, Bayes By Backpropagation, MC Dropout

Deep artificial neural networks (ANNs) are heavily overparameterized models, often with many orders of magnitude more weights than observations in the available dataset. This flexibility allows networks to minimize training error effectively, but it also creates a critical problem: model underspecification. Countless parameter configurations can explain the known data while producing radically different and unpredictable behavior on data not previously seen.

Using a single parameter configuration (a point estimate) at inference time gives rise to overconfidence. When presented with inputs far from the training distribution (out-of-distribution), a standard neural network displays unjustified confidence in its results. It may assign near-absolute certainty to its prediction—producing highly polarized probability vectors in classification or continuous estimates without error margins in regression—even in a completely unknown region of the input space, failing to convey the unreliability of the result.

To address this limitation, some studies have applied Bayesian methods to deep learning. Instead of seeking only one optimal set of weights, the Bayesian approach treats the model parameters, that is, the possible network configurations, as random variables and aims to learn their full posterior distribution $P(w \mid D)$. This change of perspective makes it possible to capture and quantify two distinct types of uncertainty: *aleatoric uncertainty*, the intrinsic and irreducible noise in the data, and *epistemic uncertainty*, the model’s lack of knowledge due to insufficient information and data.

Bayesian prediction uses marginalization, or Bayesian Model Averaging (BMA), integrating over all possible parameter configurations weighted by their posterior probabilities:

$$P(y \mid x, D) = \int_{\mathcal{W}} P(y \mid x, w) P(w \mid D) dw.$$

For deep networks, however, computing this integral exactly and deriving the true posterior are computationally intractable because of the enormous dimensionality of the parameter space. Approximation methods are therefore needed to make Bayesian principles scalable in these architectures. The following sections examine and compare two principal approaches to uncertainty estimation: Bayes by Backpropagation and Monte Carlo Dropout.

1 Bayes by Backprop

A neural network can be viewed as a probabilistic model $P(y | x, w)$ that assigns a probability to each possible response for an input using a set of weights. Those weights may be learned by maximum likelihood:

$$\begin{aligned} w^{\text{MLE}} &= \arg \max_w \log P(D | w) \\ &= \arg \max_w \sum_i \log P(y_i | x_i, w). \end{aligned}$$

Alternatively, a prior may be assigned to the weights and maximum a posteriori estimation used:

$$\begin{aligned} w^{\text{MAP}} &= \arg \max_w \log P(w | D) \\ &= \arg \max_w [\log P(D | w) + \log P(w)]. \end{aligned}$$

The posterior distribution of the weights allows predictions through an expectation, equivalent to using an uncountably large ensemble of neural networks. As noted above, high dimensionality makes the exact problem intractable. Instead, we can define a parametric family of distributions $\{q(w | \theta)\}_\theta$ and find the parameter θ^* that minimizes the Kullback–Leibler divergence¹:

$$\begin{aligned} \theta^* &= \arg \min_\theta \text{KL}[q(w | \theta) \parallel P(w | D)] \\ &= \arg \min_\theta \int q(w | \theta) \log \frac{q(w | \theta)}{P(w)P(D | w)} dw \\ &= \arg \min_\theta \text{KL}[q(w | \theta) \parallel P(w)] - \mathbb{E}_{q(w|\theta)}[\log P(D | w)] \\ &\equiv \arg \min_\theta \mathcal{F}(D, \theta). \end{aligned}$$

Reparameterization Trick

Directly minimizing the cost function $\mathcal{F}(D, \theta)$ by gradient descent is not analytically feasible for large BNNs. The principal computational obstacle is the gradient of a statistical expectation with respect to the parameters of the variational distribution itself, written as $\frac{\partial}{\partial \theta} \mathbb{E}_{q(w|\theta)}[f(w, \theta)]$. To overcome this, the Bayes by Backpropagation method introduced by Blundell et al. [1] defines a scalable Monte Carlo algorithm for variational inference. Its practical implementation uses the reparameterization trick: the stochastic weight w is expressed as a deterministic transformation $t(\theta, \epsilon)$ of a parameter-free random variable ϵ , sampled from a fixed base distribution $q(\epsilon)$, such that $q(\epsilon)d\epsilon = q(w | \theta)dw$. The expectation over w thus becomes an expectation over ϵ . Because the latter is independent of the parameters being optimized, the derivative can be moved inside the integral and the gradients computed by standard backpropagation.

$$\frac{\partial}{\partial \theta} \mathbb{E}_{q(w|\theta)}[f(w, \theta)] = \mathbb{E}_{q(\epsilon)} \left[\frac{\partial f(w, \theta)}{\partial w} \frac{\partial w}{\partial \theta} + \frac{\partial f(w, \theta)}{\partial \theta} \right].$$

For a diagonal Gaussian variational distribution over the weights, the variational parameters θ consist of means μ and variances σ^2 . To ensure a positive standard deviation during optimization, it is parameterized pointwise using the softplus function, $\sigma = \log(1 + \exp(\rho))$. Sampling an individual weight then becomes a differentiable operation:

$$w = t(\theta, \epsilon) = \mu + \log(1 + \exp(\rho)) \circ \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

¹The Kullback–Leibler divergence is defined as $D_{\text{KL}}(P \parallel Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}$ and quantifies the extra bits of information needed to encode a datum when relying on an approximate distribution rather than the exact one. It is a direct measure of the distance between the approximate model and the true distribution.

By placing the randomness in the auxiliary variable ϵ , whose distribution does not depend on θ , the derivative may be moved inside the integral. This theoretical formulation yields an iterative, computationally tractable procedure. At each training step, the algorithm proceeds as follows:

1. Generate $\epsilon \sim \mathcal{N}(0, I)$.
2. Define
 - $w = \mu + \log(1 + \exp(\rho)) \circ \epsilon$;
 - $\theta = (\mu, \rho)$;
 - $f(w, \theta) = \log q(w \mid \theta) - \log P(w)P(D \mid w)$.
3. Calculate the gradient with respect to the mean and with respect to the variance:

$$\begin{aligned}\Delta_\mu &= \frac{\partial f(w, \theta)}{\partial w} + \frac{\partial f(w, \theta)}{\partial \mu}, \\ \Delta_\rho &= \frac{\partial f(w, \theta)}{\partial w} \frac{\epsilon}{1 + \exp(-\rho)} + \frac{\partial f(w, \theta)}{\partial \rho}.\end{aligned}$$

4. Update the variational parameters:

$$\begin{aligned}\mu &\leftarrow \mu - \alpha \Delta_\mu, \\ \rho &\leftarrow \rho - \alpha \Delta_\rho.\end{aligned}$$

The term $\frac{\partial f(w, \theta)}{\partial w}$ is the gradient calculated by the usual backpropagation algorithm. Bayes by Backprop is thus computationally compatible with deep-learning frameworks and permits optimization of variational parameters with unbiased Monte Carlo gradients.

Choice of Prior

Bayes by Backprop removes the requirement for closed mathematical forms in inference, allowing flexible choices of prior distribution. One particularly effective formulation in this setting is the *scale-mixture prior*, defined as a mixture of two Gaussian densities:

$$P(\mathbf{w}) = \prod_j \pi \mathcal{N}(w_j \mid 0, \sigma_1^2) + (1 - \pi) \mathcal{N}(w_j \mid 0, \sigma_2^2),$$

where w_j is the j -th network weight and the variances are chosen so that $\sigma_1 > \sigma_2$ and $\sigma_2 \ll 1$.

Conceptually, this distribution resembles a spike-and-slab prior: the first component (the “slab”) provides heavy tails, allowing some weights to take substantial values; the second (the “spike”) concentrates considerable probability mass at zero, pushing most weights toward zero.

Unlike a traditional spike-and-slab approach, here the mixture parameters are shared by all weights and held fixed during training (typically selected in advance by cross-validation). This design choice has two advantages: it makes optimization by stochastic gradient descent more tractable and avoids the need, and associated instability risks, to optimize the prior parameters directly on the training data.

2 MC Dropout

Dropout is traditionally used as a regularization mechanism in neural networks to reduce overfitting during training. Gal et al., however, extended it to inference: rather than disabling randomness at test time, they proposed making multiple stochastic forward passes through the network and aggregating their results to estimate the predictive mean and variance. This

technique, known as Monte Carlo Dropout, rests on the result (shown by Gal et al. [2]) that a neural network of arbitrary depth and nonlinearity, with dropout applied before each weight layer, implicitly performs variational inference. In particular, this architecture is mathematically equivalent to a Bayesian approximation of a deep Gaussian process (DGP).

In this setting, consider the usual cost function minimized during training:

$$\mathcal{L}_{\text{dropout}} = \frac{1}{N} \sum_{i=1}^N E(y_i, \hat{y}_i) + \lambda \sum_{i=1}^L \left(\|W_i\|_2^2 + \|b_i\|_2^2 \right),$$

where:

- N is the number of observations in the dataset;
- y_i is the observed target, and $\hat{y}_i$ the network prediction for input x_i ;
- $W_i \in \mathbb{R}^{K_i \times K_{i-1}}$ and $b_i \in \mathbb{R}^{K_i}$ are the weight matrix and bias vector for layer i ;
- λ is the weight-decay penalty factor.

Optimizing this function effectively corresponds to maximizing $\mathcal{F}(D, \theta)$ (called the Evidence Lower Bound). The data-error term acts as an expected log-likelihood, while the weight penalty corresponds to minimizing the Kullback–Leibler divergence between the variational distribution generated by stochastic dropout and the deep Gaussian-process prior.

Regression Model

In regression tasks, the likelihood of the observed data is assumed to be Gaussian. For an input x and weights w , the output y is modeled as

$$P(y \mid x, w) = \mathcal{N}(y; \hat{y}(x, w), \tau^{-1}I).$$

Here $\hat{y}(x, w)$ is the neural network’s predictive function, and $\tau = \frac{p}{2N\lambda}$, with p the dropout probability, is the model precision parameter. Predictions are obtained by averaging T stochastic forward passes:

$$y^* = \frac{1}{T} \sum_{t=1}^T \hat{y}(x, (\widehat{W}_1^t, \dots, \widehat{W}_L^t)).$$

The model uncertainty (predictive variance) in regression is then estimated by combining the sample variance of the T forward passes (epistemic uncertainty) with the inverse model precision (aleatoric uncertainty):

$$\text{Var}(y^*) \approx \tau^{-1}I + \frac{1}{T} \sum_{t=1}^T (\hat{y}_t^*)^\top (\hat{y}_t^*) - \mathbb{E}(y^*)^\top \mathbb{E}(y^*).$$

Classification Model

For classification, the network output is usually a probability vector obtained by applying softmax. As in regression, predictive probabilities are calculated by collecting the softmax outputs of T stochastic passes and averaging them for each class:

$$p(y \mid x) \approx \frac{1}{T} \sum_{t=1}^T \text{softmax}(\hat{y}_t^*).$$

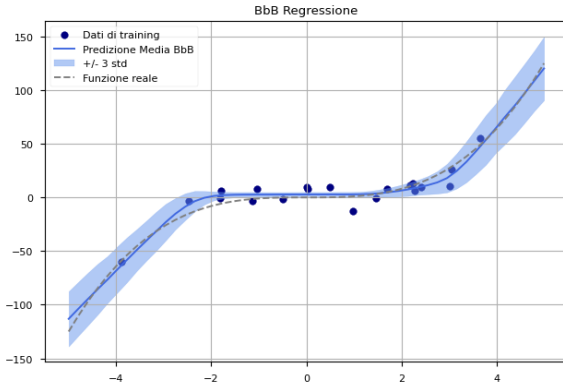
Uncertainty can be quantified by examining the resulting probability distributions. The indicators are:

- **Predictive entropy:** If the mean probability vector has high entropy (that is, the probabilities are nearly uniform across classes), the model is uncertain.
- **Softmax variation:** Examine how much class assignments change among Monte Carlo passes. If the model predicts different classes for the same input on different passes, uncertainty is high.

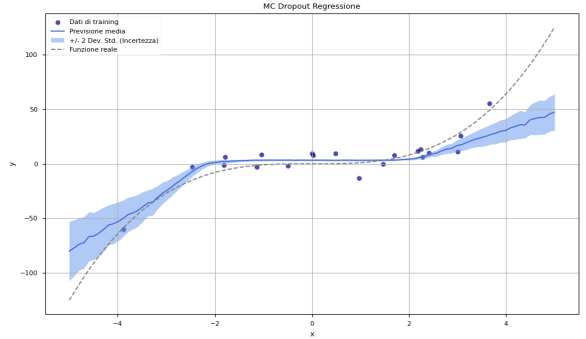
3 Numerical Analysis

Regression

To qualitatively assess the ability of the two methods to estimate uncertainty, a synthetic dataset was generated from the true function $y = x^3$. Twenty points were sampled uniformly from the interval $[-4, 4]$, and Gaussian noise sampled from $\mathcal{N}(0, 9)$ was added.



(a) Result with Bayes by Backprop



(b) Result with Monte Carlo Dropout

Figure 1: Comparison of uncertainty estimates in regression. Original artwork labels: *Dati di training* = training data; *Previsione media* = mean prediction; *Funzione reale* = true function; *Dev. Std. (Incertezza)* = standard deviation (uncertainty). The uncertainty bands are labeled ± 3 standard deviations in (a) and ± 2 in (b).

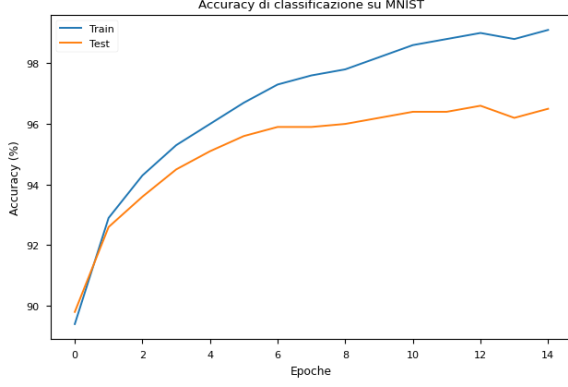
Comparing the resulting plots, both models are expected to approximate the mean correctly at the observed points. The continuous nature of BbB’s variational distribution, however, tends to produce a much smoother and better-calibrated widening of variance away from the data domain, while MCD, being based on a discrete estimate, may produce a slightly different range of predictions and, in some cases, less uniform divergence.

Classification and Out-of-Distribution (OOD) Detection

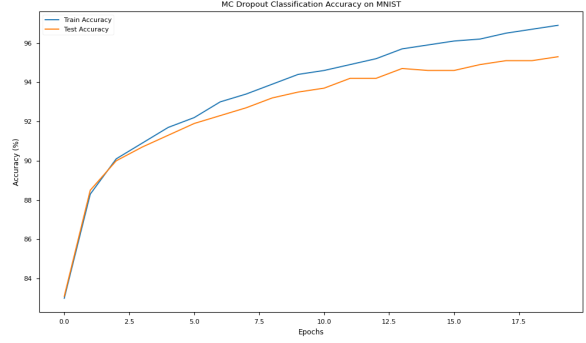
For classification tasks, the analysis turned to real visual datasets. The models were first trained and tested on MNIST (in-distribution data) and then evaluated on Fashion-MNIST, used only at test time as an out-of-distribution (OOD) dataset. This approach assesses whether the network can express its lack of knowledge when faced with previously unseen inputs.

Important structural differences between the methods emerge during training. MCD typically needs more epochs to converge, a delay attributed to the high variance introduced by stochastic masking of the weights (dropout itself) on every forward pass. Computation time per epoch, however, strongly favors MCD. BbB requires twice as many parameters (for each network weight it must learn both a mean μ and a standard deviation ρ) and is computationally more demanding. Thus, while the two methods are comparable in practice for shallow architectures, for deep-learning architectures MCD becomes the only genuinely scalable approach of the two.

Accuracy and uncertainty metrics at inference show distinct behavior:



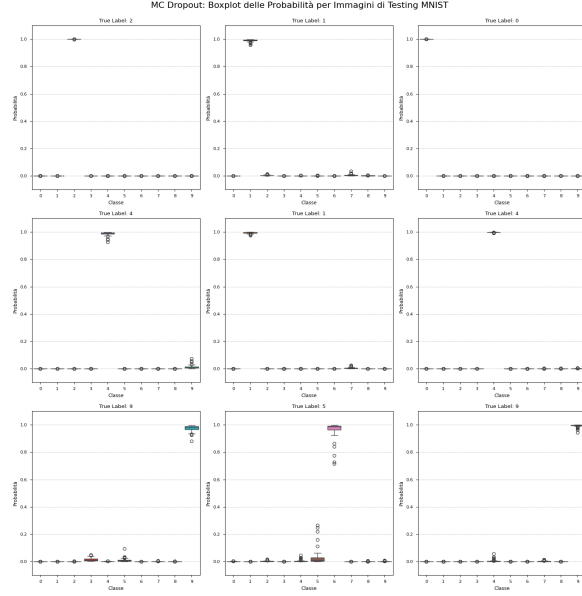
(a) Accuracy with Bayes by Backprop



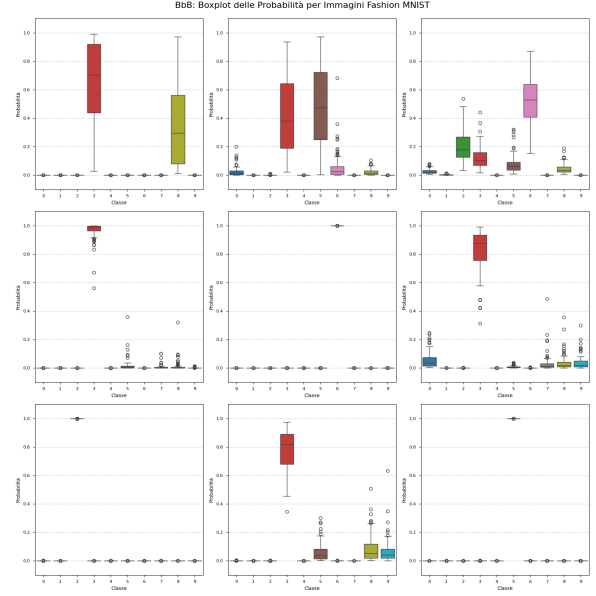
(b) Accuracy with Monte Carlo Dropout

Figure 2: Comparison of MNIST accuracy curves. The Italian labels in (a), *Accuratezza di classificazione su MNIST* and *Epoche*, mean “classification accuracy on MNIST” and “epochs.”

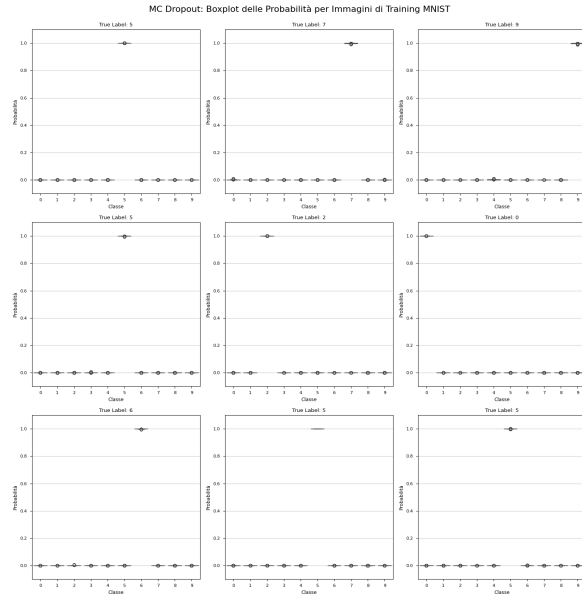
- **In-distribution (MNIST):** Both methods are accurate and highly confident. When they evaluate MNIST test images (from the same distribution as training), probabilities correctly collapse onto the true class, giving predictive entropy very close to zero.
- **Out-of-distribution (Fashion-MNIST):** When presented with clothing images, Bayesian models are expected to produce flat probability vectors (for example, approximately 10% per class) and maximal entropy. Simplified approximate distributions, however, prevent a perfect representation of uncertainty and leave some residual overconfidence. OOD samples also often share geometric patterns with the original training data. Consequently, softmax, which exponentially amplifies variations in latent scores, leads the model to “recognize” these partial similarities. This polarizes the final prediction, preventing the system from returning the perfectly flat (10%) probability distribution that would be expected under complete uncertainty.



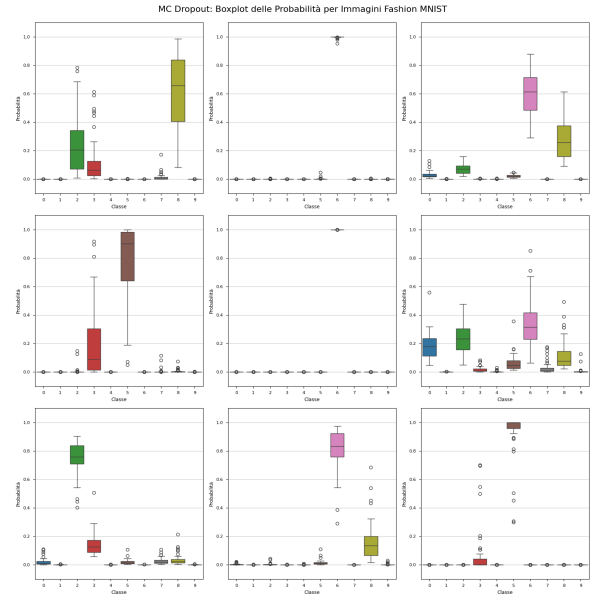
(a) BbB on MNIST



(b) BbB on Fashion-MNIST



(c) MCD on MNIST



(d) MCD on Fashion-MNIST

Figure 3: Comparison of uncertainty distributions (box plots) for BbB and MCD on MNIST and Fashion-MNIST. Original artwork labels: *Probabilità* = probability; *Classe* = class; *Immagini di Training* = training images. The title inside panel (a) says “MC Dropout” although its printed subcaption identifies it as BbB; both labels are retained from the source.

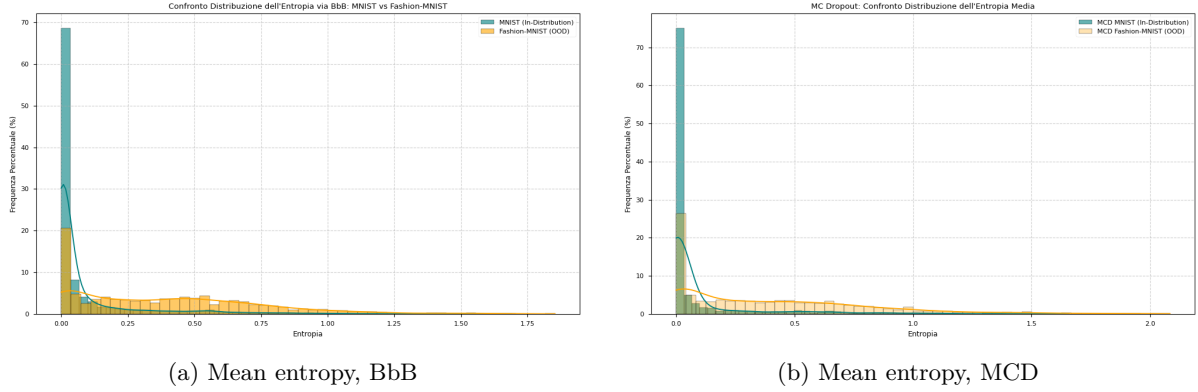


Figure 4: Comparison of mean predictive entropy for Bayes by Backprop and Monte Carlo Dropout. Original artwork labels: *Confronto distribuzione dell'entropia* = comparison of entropy distributions; *Frequenza Percentuale* = percentage frequency; *Entropia* = entropy.

As shown by the box plots and entropy distributions, the BbB model provides significantly more precise and better-calibrated uncertainty estimates than MCD. Bayes by Backprop allocates substantially higher and more compact mean entropy to OOD samples. In contrast, the MCD approximation, although able to signal the anomaly, sometimes displays tails of excessive confidence (residual overconfidence). This is attributed to the nature of MCD’s variational approximation: representing uncertainty with a mixture of Bernoulli variables (on/off) is a less rich and expressive posterior approximation than learning an independent continuous Gaussian distribution for every weight, as in BbB.

References

- [1] Charles Blundell et al. “Weight uncertainty in neural network.” In: *International Conference on Machine Learning*. PMLR, 2015, pp. 1613–1622.
- [2] Yarin Gal and Zoubin Ghahramani. “Dropout as a Bayesian approximation: Representing model uncertainty in deep learning.” In: *International Conference on Machine Learning*. PMLR, 2016, pp. 1050–1059.
- [3] Laurent Valentin Jospin et al. “Hands-on Bayesian neural networks—A tutorial for deep learning users.” In: *IEEE Computational Intelligence Magazine* 17.2 (2022), pp. 29–48.
- [4] Andrew G Wilson and Pavel Izmailov. “Bayesian deep learning and a probabilistic perspective of generalization.” In: *Advances in Neural Information Processing Systems* 33 (2020), pp. 4697–4708.
- [5] Alex Graves. “Practical variational inference for neural networks.” In: *Advances in Neural Information Processing Systems* 24 (2011).